

Representation and Time–Memory Tradeoffs in Symmetric Cryptography: Finite-Field Basis Transformations for AES and Reproducible Hellman Experiments

Ali Mkhida
Algorizk Labs

Working research note, 2026

Abstract

This note upgrades a historical coursework project on high-performance cryptography into a reproducible research artifact. The original presentation combined finite-field representation changes for AES-like arithmetic with practical secret-key recovery experiments based on exhaustive search, Hellman time–memory tradeoffs, and distinguished points. The repository now preserves the original presentation as a historical PDF, while this note gives a separate ePrint-style reconstruction.

The first part of the note records the representation mathematics: polynomial and normal bases of $\mathbb{F}(2^n)$, basis-change maps, field isomorphisms, composite-field inversion, the role of the parameter δ , and the non-LUT view of the AES S-box. The second part gives a modern reproducible implementation of the cryptanalytic experiments over deliberately reduced DES state spaces. The main empirical observation is that, for the tested single-map Hellman construction with $mt/N = 1$, the median exact table coverage is only about 16.8%, far below the independent-sample occupancy reference $1 - e^{-1} \simeq 63.2\%$. The gap is explained by functional-graph coalescence: chain states are not independent samples but iterates of one fixed map.

1 Historical artifact and updated goal

The original project was titled *High Performance Computing for Cryptography*. That title correctly described the course context, but it does not reveal the mathematical object studied by the modernized repository. The updated repository is organized around two related themes:

1. finite-field representation changes for efficient AES-like arithmetic;
2. cryptanalytic time–memory tradeoffs for reduced key spaces.

The historical presentation remains preserved unchanged as `docs/original-presentation.pdf`. This document is separate: it is the research-note version, intended to evolve toward an ePrint-style manuscript. Where the original slides were visual and compressed, this note makes the algebraic objects explicit and separates historical claims from newly reproduced measurements.

2 Finite fields in characteristic two

Let $n \geq 1$. The binary extension field $\mathbb{F}(2^n)$ may be represented as

$$\mathbb{F}(2^n) \simeq \mathbb{F}(2)[X]/(p(X)),$$

where p is an irreducible polynomial of degree n . If α denotes the class of X , the *polynomial basis* is

$$(1, \alpha, \alpha^2, \dots, \alpha^{n-1}).$$

Every field element is written uniquely as

$$a = a_0 + a_1\alpha + \dots + a_{n-1}\alpha^{n-1}, \quad a_i \in \mathbb{F}(2).$$

A second important representation is a *normal basis*. An element $\beta \in \mathbb{F}(2^n)$ is normal over $\mathbb{F}(2)$ if

$$\mathcal{N}(\beta) = (\beta, \beta^2, \beta^{2^2}, \dots, \beta^{2^{n-1}})$$

is a basis of $\mathbb{F}(2^n)$ over $\mathbb{F}(2)$. In this representation, the Frobenius map

$$\varphi : x \mapsto x^2$$

acts as a cyclic shift on coordinates:

$$\sum_{i=0}^{n-1} b_i \beta^{2^i} \mapsto \sum_{i=0}^{n-1} b_i \beta^{2^{i+1}}.$$

This is the central reason normal bases are attractive in hardware: squaring can be implemented by rewiring rather than by a general multiplier.

3 Basis-change maps

Let

$$P = (1, \alpha, \dots, \alpha^{n-1})$$

be the polynomial basis and

$$N = (\beta, \beta^2, \dots, \beta^{2^{n-1}})$$

be a normal basis. Each normal basis vector can be expanded in the polynomial basis:

$$\beta^{2^j} = \sum_{i=0}^{n-1} B_{ij} \alpha^i, \quad B_{ij} \in \mathbb{F}(2).$$

The matrix $B = (B_{ij})$ maps normal-basis coordinates to polynomial-basis coordinates:

$$[a]_P = B[a]_N.$$

If B is invertible, the inverse map is

$$[a]_N = B^{-1}[a]_P.$$

This is the abstract form of the “canonical basis to normal basis” part of the original presentation. In a hardware implementation, these matrices are binary linear maps. They can be implemented as XOR networks. Their cost is therefore measured by XOR count, depth, fanout, and routing, rather than by field multipliers.

3.1 Multiplication tensors

A basis determines how multiplication is implemented. If $e_0, \dots, e_{n-1}$ is a basis, define structure constants $c_{ij}^k \in \mathbb{F}(2)$ by

$$e_i e_j = \sum_{k=0}^{n-1} c_{ij}^k e_k.$$

For

$$a = \sum_i a_i e_i, \quad b = \sum_j b_j e_j,$$

the product coordinates are

$$(ab)_k = \sum_{i,j} a_i b_j c_{ij}^k.$$

In polynomial basis, reduction modulo $p(X)$ gives these constants. In a normal basis, squaring is cheap, but multiplication may require a nontrivial bilinear network. Representation engineering is therefore a tradeoff: choose a basis that makes the dominant operation cheap, and pay only controlled basis-conversion cost where necessary.

4 Field isomorphisms

Suppose two degree- n representations are given:

$$A = \mathbb{F}(2)[X]/(f(X)), \quad B = \mathbb{F}(2)[Y]/(g(Y)).$$

Let $\bar{X}$ be the image of X in A . A field isomorphism $\psi : A \rightarrow B$ is determined by the image of $\bar{X}$. If $\theta \in B$ satisfies

$$f(\theta) = 0,$$

then there is a unique homomorphism

$$\psi(h(\bar{X})) = h(\theta).$$

Because both fields have 2^n elements, a nonzero homomorphism is an isomorphism.

Thus, finding an isomorphism is reduced to finding a root of the defining polynomial of one representation inside the other representation. Once θ is known, the basis-change matrix is obtained from

$$1, \theta, \theta^2, \dots, \theta^{n-1}$$

written in the target basis.

This is the mathematical content behind the “finding isomorphisms” portion of the historical slides. It is also the bridge between an implementation written in the AES polynomial basis and one using normal or composite bases.

5 AES field arithmetic and non-LUT S-box computation

AES represents bytes as elements of

$$\mathbb{F}(2)[X]/(m(X)), \quad m(X) = X^8 + X^4 + X^3 + X + 1.$$

The AES S-box maps a byte a to

$$S(a) = A(a^{-1}) + c,$$

where 0^{-1} is defined as 0, A is a fixed affine map over $\mathbb{F}(2)$, and c is a fixed byte constant.

A table implementation stores $S(a)$ for all 256 inputs. A non-LUT implementation instead computes:

$$a \xrightarrow{\text{basis change}} \tilde{a} \xrightarrow{\text{field inversion}} \tilde{a}^{-1} \xrightarrow{\text{basis change}} a^{-1} \xrightarrow{\text{affine map}} S(a).$$

The advantage is that the large lookup table is replaced by structured XOR and small-field arithmetic. This is useful in hardware contexts where table memory, side-channel behavior, or datapath regularity matter.

6 Composite fields and the role of δ

A common way to compute inversion in $\mathbb{F}(2^8)$ is to represent it as a quadratic extension of $\mathbb{F}(2^4)$:

$$\mathbb{F}(2^8) \simeq \mathbb{F}(2^4)[U]/(U^2 + U + \delta),$$

where $\delta \in \mathbb{F}(2^4)$ is chosen so that $U^2 + U + \delta$ is irreducible. Every element has the form

$$z = a + bU, \quad a, b \in \mathbb{F}(2^4).$$

Because $U^2 + U + \delta = 0$, the conjugate of U over $\mathbb{F}(2^4)$ is $U + 1$. Therefore the conjugate of z is

$$\bar{z} = a + b(U + 1) = (a + b) + bU.$$

The norm is

$$N(z) = z\bar{z} = (a + bU)(a + b(U + 1)) = a^2 + ab + \delta b^2.$$

Hence, for $z \neq 0$,

$$z^{-1} = \frac{\bar{z}}{N(z)} = \frac{(a + b) + bU}{a^2 + ab + \delta b^2}.$$

This reduces inversion in $\mathbb{F}(2^8)$ to:

1. squaring in $\mathbb{F}(2^4)$;
2. multiplication in $\mathbb{F}(2^4)$;
3. one inversion in $\mathbb{F}(2^4)$;
4. multiplication by the fixed constant δ .

The historical presentation's section on speeding up computations involving δ belongs precisely here. Multiplication by a fixed element is a linear map over $\mathbb{F}(2)$. If

$$L_\delta : x \longmapsto \delta x$$

is precomputed as a 4×4 binary matrix, then δb^2 can be implemented as a small XOR network after the cheap squaring map in $\mathbb{F}(2^4)$. In hardware terms, δ is not a general multiplier in the critical path; it is a fixed linear transform.

7 From representation arithmetic to cryptanalytic experiments

The finite-field part of the project concerns the cost of one encryption datapath. The cryptanalytic part concerns the cost of searching over many candidate keys. These are linked but distinct:

- finite-field basis choices change the cost of encryption rounds;
- time–memory tradeoffs change the cost profile of key recovery.

The original coursework connected these views by considering secret-key recovery experiments after the AES arithmetic discussion. The modern repository makes the second part reproducible over deliberately reduced DES state spaces. DES is obsolete and is used here only because it provides a small, standard block cipher setting suitable for reproducible experiments.

8 Reduced DES state model

The historical Java prototype varied four raw DES bytes. However, a DES key is encoded in eight bytes with one parity bit per byte. Thus four varying raw bytes do not automatically give 32 independent effective DES key bits.

The modern model defines a reduced state space explicitly:

$$X_b = \{0, 1, \dots, 2^b - 1\}, \quad 1 \leq b \leq 28.$$

A state $x \in X_b$ is mapped to a canonical DES key encoding $K(x)$ with odd parity. The first four DES bytes are fixed, and the final four bytes carry up to seven effective bits each. This gives at most 28 effective variable bits.

For a fixed plaintext P , the encryption oracle is

$$C_x = \text{Enc}_{K(x)}(P).$$

A reduction function

$$R : \{0, 1\}^* \rightarrow X_b$$

maps ciphertexts back into reduced states. The core functional graph is

$$F(x) = R(\text{Enc}_{K(x)}(P)).$$

Hellman chains and distinguished-point chains are both walks in this directed functional graph.

9 Exhaustive search

Given a target ciphertext C^* , exhaustive search tests every state:

$$\text{Enc}_{K(x)}(P) \stackrel{?}{=} C^*.$$

It requires no offline table and no stored endpoints. Its memory cost is constant, but its online cost is $O(2^b)$ encryptions in the worst case.

In the verified experiment with $b = 16$, exhaustive search recovers all targets because it covers the entire reduced state space by definition.

10 Hellman chains

A Hellman table is generated by choosing start states

$$s_1, \dots, s_m \in X_b$$

and iterating F for a fixed chain length t :

$$s_i, F(s_i), F^2(s_i), \dots, F^t(s_i).$$

Only the pair

$$(s_i, F^t(s_i))$$

is stored. The second component is the endpoint.

For a target ciphertext C^* , the online phase computes

$$R(C^*), \quad F(R(C^*)), \quad F^2(R(C^*)), \dots$$

and checks whether one of these projected endpoints appears in the table. When an endpoint matches, the corresponding stored chain is regenerated from its start point and the target ciphertext is verified directly.

The modern implementation stores endpoints in a hash index rather than repeatedly scanning every endpoint. It also retains endpoint collisions and regenerates all candidate chains, because different starts may lead to the same endpoint.

10.1 Why the independent occupancy heuristic is optimistic

If mt states were sampled independently and uniformly from a universe of size $N = 2^b$, the expected covered fraction would be

$$1 - \left(1 - \frac{1}{N}\right)^{mt} \approx 1 - e^{-mt/N}.$$

But a Hellman table does not draw mt independent samples. It samples start points, then applies the same function F repeatedly. If two chains meet, they coalesce forever:

$$F^a(s_i) = F^b(s_j) \implies F^{a+r}(s_i) = F^{b+r}(s_j) \quad \text{for all } r \geq 0.$$

Thus the functional-graph structure of F can reduce actual coverage far below the independent-sampling reference.

This is not a measurement anomaly; it is a structural effect. It is also why exact coverage analysis is included in the repository rather than relying on the reference value in Eq. (10.1).

11 Distinguished points

A distinguished-point method stores only chains that reach an easily tested subset $D \subset X_b$. In the modern implementation,

$$D_d = \{x \in X_b : x \bmod 2^d = 0\}.$$

Equivalently, the lowest d state bits are zero. Under a uniform-state heuristic, the expected hitting time is approximately 2^d transitions.

A chain starts at s_i and is followed until either:

1. it reaches a distinguished point; or
2. it exceeds a maximum chain length.

Only successful chains are stored. Chains that do not hit a distinguished point before the cutoff are counted as truncated. During lookup, the target is projected forward until it reaches a distinguished point or the bound is exceeded. Candidate chains sharing that endpoint are regenerated and verified against the target ciphertext.

12 Implementation corrections

The modern implementation changes the historical prototype in several ways:

1. The reduced key space counts effective DES key bits, not raw bytes.
2. Endpoints are indexed rather than searched by nested linear scans.
3. Candidate chains are always verified against the target ciphertext.
4. Distinguished-point chains are bounded and truncated chains are counted.
5. The implementation records exact state coverage outside the timed path.
6. All experiments are generated from explicit seeds.

These changes make the code a reference implementation rather than a one-off demonstration. They also separate algorithmic claims from JVM timing noise.

13 Verified experiment

The committed benchmark uses:

- effective state dimension $b = 16$, so $N = 65\,536$;
- 30 deterministic seeded trials;
- Hellman parameters $m = 1024$, $t = 64$;
- distinguished-point parameters 1024 starts, $d = 6$, and maximum chain length 256;
- a separate warm-up run before collecting timings;
- exact unique-state table coverage computed outside the timed path.

Method	Exact recovery	Median exact coverage	Median online time
Exhaustive	30/30	100.0%	25.863 ms
Hellman	4/30	16.8%	3.399 ms
Distinguished points	3/30	15.4%	1.790 ms

For Hellman, the parameters satisfy

$$\frac{mt}{N} = \frac{1024 \cdot 64}{65\,536} = 1.$$

The independent-sample occupancy reference would therefore be

$$1 - e^{-1} \simeq 63.2\%.$$

The exact measured median coverage is instead 16.8%. This confirms that the fixed-map functional graph is the right mathematical model for these tables.

For distinguished points, the expected waiting-time reference is

$$2^d = 64$$

transitions. Across the 30 tables, the median exact stored-table coverage is 15.4%, and many starts merge onto the same distinguished endpoint.

14 Toward a Rust implementation

The Java implementation is intentionally kept as the current reference semantics. The next phase should not be a direct performance rewrite without cross-checks. A Rust implementation should first reproduce:

1. the same reduced DES state encoding;
2. the same key parity convention;
3. the same reduction function;
4. the same chain endpoints for fixed seeds;
5. the same exact coverage results;
6. the same recovery decisions on committed test vectors.

Only after cross-language equivalence is established should performance work begin. The natural Rust roadmap is:

Java reference $\longrightarrow$ test vectors $\longrightarrow$ Rust reference $\longrightarrow$ equivalence tests $\longrightarrow$ parallel and SIMD optimization.

Potential Rust tools include property-based testing, Criterion-style benchmarking, and parallel chain generation. A future hardware path would treat encryption and reduction as pipeline kernels, but that is outside the current repository.

15 Future research directions

Several directions follow naturally from this update.

Multiple reductions. Classical Hellman tables often use table-dependent reductions. The current implementation uses one fixed reduction. Introducing indexed reductions R_j would reduce chain merging and make the comparison with classical tradeoff theory more direct.

Rainbow and multi-table variants. Rainbow tables change the reduction per column. A reproducible comparison between fixed-reduction Hellman tables, multiple Hellman tables, and rainbow tables would clarify how much of the measured coverage loss is due to functional-graph coalescence.

Parameter optimization. The current benchmark is deliberately small. A larger study should sweep m , t , distinguished-point difficulty d , and chain bounds, reporting coverage, online work, false candidates, and memory.

Finite-field arithmetic implementation. The AES finite-field part should be turned into executable code: basis-change matrices, composite-field inversion, δ multiplication, and non-LUT S-box circuits. The code should report XOR count, depth, and test equivalence against the AES S-box.

Rust and hardware. Rust should first match Java semantics and only then pursue speed. Hardware work should be based on stable test vectors and should separate AES arithmetic optimization from time-memory tradeoff table generation.

16 Conclusion

The updated repository separates historical provenance from modern reproducibility. The original presentation remains intact, while this note reconstructs the finite-field mathematics and connects it to a tested time-memory tradeoff implementation.

The most important experimental lesson is that Hellman table size alone does not determine coverage. For the tested fixed reduction, $mt/N = 1$ but exact coverage is only about 16.8%. The correct model is the functional graph of $F(x) = R(\text{Enc}_{K(x)}(P))$, where chain coalescence is structural. This makes the project a useful base for a future Rust implementation and for a broader study of reductions, table families, and cryptographic datapath representations.

References

- [1] M. E. Hellman. A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980.
- [2] P. C. van Oorschot and M. J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12:1–28, 1999.
- [3] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*, FIPS 197, updated 2023.
- [4] J. Daemen and V. Rijmen. *The Design of Rijndael: AES – The Advanced Encryption Standard*. Springer, 2002.
- [5] D. Canright. A very compact S-box for AES. In *Cryptographic Hardware and Embedded Systems – CHES 2005*, pages 441–455. Springer, 2005.
- [6] A. Satoh, S. Morioka, K. Takano, and S. Munetoh. A compact Rijndael hardware architecture with S-box optimization. In *Advances in Cryptology – ASIACRYPT 2001*, pages 239–254. Springer, 2001.